R6-1 Finding Independent Set 分数 8

全屏浏览 作者 陈越 单位 浙江大学

Given a simple undirected graph $G = (V, E)$. An **independent set** of G is a set $S \subseteq V$ such that no two members of S are connected by an edge in E. Finding the maximum independent set of G is an NP-hard problem. Here you are supposed to implement a greedy hueristic to find a near-maximum independent set.

The algorithm works in the following way:

1. Collect any one unvisited vertex v into S.
2. Delete all the vertices (and all the edges incident on them) that are adjacent to v from G.
3. Repeat steps 1 and 2 until there is no unvisited vertex left in G.

In order to obtain the unique solution, when there are many options in step 1, you must always choose the vertex with the smallest index.

Format of functions:

```
int Independent_Set( MGraph G, int S[] );
```

Here `MGraph` is defined as the following:

```
typedef struct GNode *PtrToGNode;
struct GNode{
    int Nv;  /* number of vertices */
    int Ne;  /* number of edges   */
    int G[MaxVertexNum][MaxVertexNum]; /* adjacency matrix */
};
typedef PtrToGNode MGraph;
```

where `MaxVertexNum` is a constant defined by the judge.

The indices of the vertices in the resulting independent set is stored in the array `int S[]`. Note that the vertices are indexed from 1 to the number of vertices `Nv`. However, the indices of the adjacency matrix `G` starts from 0.

The function `Independent_Set` is supposed to follow the above algorithm to produce an independent set, save the results in `S`, and return the number of vertices in `S`.

Sample program of judge:

```
#include <stdio.h>
#include <stdlib.h>

#define MaxVertexNum 1000
```

```
typedef struct GNode *PtrToGNode;
struct GNode{
    int Nv;  /* number of vertices */
    int Ne;  /* number of edges   */
    int G[MaxVertexNum][MaxVertexNum]; /* adjacency matrix */
};
typedef PtrToGNode MGraph;

MGraph Build_Graph(); /* implemented by judge */
int Independent_Set( MGraph G, int S[] );

int main()
{
    int i, n, S[MaxVertexNum];
    MGraph G;

    G = Build_Graph();
    n = Independent_Set(G, S);
    printf("%d", S[0]);
    for (i=1; i<n; i++) printf(" %d", S[i]);

    return 0;
}
/* Your function(s) will be put here */
```

Sample Input:

```
8 7
1 5
5 4
4 2
2 3
3 6
6 1
6 2
```

Sample Output:

```
1 2 7 8
```

Note:

This method can help us find an independent set, but not necessarily a **maximum** independent set. For example, the maximum solution of the sample graph is {1, 3, 4, 7, 8}.

代码长度限制	16 KB
时间限制	400 ms
内存限制	64 MB

C (gcc)

```c
int Independent_Set( MGraph G, int S[] )
{
    int top = -1,i;
    int* flag = (int*)malloc((1+G->Nv) * sizeof(int));
    for(i = 1;i <= G->Nv;i ++)
    {
        flag[i] = 0;
    }
   while(1)
   {
       int index = 1;
       while(index <= G->Nv && flag[index] != 0) index ++;
       if(index > G->Nv) break;
       flag[index] = 1;
       S[++top] = index;
       for(i = 0;i < G->Nv;i ++)
           if(G->G[index-1][i] != 0) flag[i+1] = 1;
   }
    return top + 1;
}
```

测试用例

查看上次提交